

Prompting in an Ensemble Way: A Study of Prompt Ensemble in Continuous and Discrete Space

Yiming Tang NingHao He Ruyuan Zuo Zhaoqi Xiao
{tangyimi, ninghaoh, rzuo, zhaoqixi}@usc.edu

Abstract

Language models (LMs) have recently shown remarkable few-shot performance with the GPT-3 (Brown et al., 2020a) model. To further explore this area, this project aims to investigate the few-shot performance on pre-trained LMs in various tasks. Key questions we have addressed include whether LMs can demonstrate decent performance compared to GPT-3, to what extent natural language descriptions or prompts can improve their few-shot performance, and how to design effective prompts for LMs in few-shot tasks. We show this research in two steps: a) We explore discrete prompt and continuous prompt generation and provide that the continuous prompt performs better than the discrete prompt. b) We test that an ensemble method in the embedding space can significantly help task performance.

1 Introduction

The GPT-3 model has demonstrated impressive few-shot learning capabilities, sometimes over-human performance. In Nature Language Understanding tasks, GPT-3 can predict language understanding tasks with only a natural language prompt and a few demonstrations of the task. Raising ChatGPT has changed the gameplay completely and makes prompt LM an efficient way to elicit knowledge from Pretrained Language Models (PLM).

However, due to its massive size and expensive usage fee, it is challenging to use in practical applications. We focus on a more realistic scenario where we have access to a moderately sized language model and only a small number of examples in a few-shot setting. Finding and implementing appropriate prompts that lead to better performance and few-shot settings are realistic and efficient to train on. We also show that while keeping the model pre-trained parameters frozen, we drastically reduced the training time. Thus, using a prompt can achieve similar performance by fine-tuning a full

model for the model for downstream tasks. Thus, We need a parameter-efficient way of modifying PLM, which we focus on as prompting.

Similar to humans, to learn a task, they don't need to see many examples to learn to do a task. Instead, models are provided with a few examples, which we call a few shots, that can tell the model what to do with little or no examples. This is especially powerful for GPT model (Brown et al., 2020a; Ye et al., 2023). Here we focus on a few shot settings for the prompt because we want to probe smaller models to achieve near-human performance.

Discrete prompts are specific inputs designed to guide language models toward desired outputs. Their main advantage is the ability to focus a model's attention on specific aspects of a task, yielding more accurate and contextually relevant responses without full fine-tune. However, making effective discrete prompts can be challenging, often requiring trial-and-error or external knowledge.

The continuous prompt has more expressive power to discrete an soft prompt that better than discrete. Rather than forcing a specific word token, soft-prompt lives in continuous vector space. Not only does this make it easier for the model to optimize and do gradient update, it also relaxes the constraint for the prompt and allows a soft prompt to learn in between the discrete space.

As a tradition ensemble models seems to achieve the best of all models (Clark et al., 2019). Although several article has discussed various way to ensemble soft prompt, we want to use a attention based method because attentions has shown to be useful focus on the most relevant parts of the input sequence when making predictions.

The main research questions of this projects are as follows:

1) Few-shot performance? LMs can demonstrate decent performance compared to GPT-3: smaller LMs with fewer parameters can also achieve im-

pressive results on some NLP tasks, depending on their architecture and training data.

2) To what extent prompts can improve their few-shot performance?

3) How to design effective prompts for LMs in few-shot tasks.

4) Ensemble prompts with different ways, show that Attention based Ensemble works best

2 Related Work

Few-shot learning Few-shot learning is derived from human learning that human can generally conduct a new language task with a few examples given. Pre-trained language models (LMs) are inherently few-shot learners (Brown et al., 2020b). They can perform language tasks given a few examples without any gradient updates, but with text interaction between LMs and few text demonstrations. Reynolds and McDonell (2021) proposed demonstrations help LMs locate an already-learned task during pre-training instead of teaching a new task.

P-tuning Liu et al. (2021) proposed a new method called P-tuning, which employs trainable continuous prompt embedding. They discover that P-tuning also improves BERTs' performance in both few-shot and supervised settings while largely reducing the need for prompt engineering. Instead of using discrete prompt, they proposed to continuous prompts. The P-tuning replaces the input embedding of pre-trained language models with its differential output embedding and automatically searches better prompts in the continuous space. By doing so, they are able to augment pre-trained model's ability in natural language understanding by automatically searching better prompts in the continuous space. One of the key advantages of P-tuning is that it reduces the dependence on a large validation set, minimizes the impact of adversarial prompts, and mitigates over-fitting.

Prompt generation Gao et al. (2020) uses prompt-based tuning generate prompts both manually and automatically. They mask out the sentence and fine-tune T5 (Raffel et al., 2019a) and the fine-tune T5 will generate better prompts. They use SBERT (Reimers and Gurevych, 2019) to sample the most similar demonstrations, concatenating them all with the input. It shows a good few-shot performance on RoBERTa (Liu et al., 2019). Lester et al. (2021) conducts prompt-tuning, which ba-

sically the same prompt generation method using T5 but adding Sentinel to the downstream targets. They also add "LM Adaptation" but remodel the objective discussed by Raffel et al. (2019b), which the model generates text given a natural prefix. They also consider prompt-tuning ensemble by taking the majority vote. Shin et al. (2020) demonstrates a viable parameter-free alternative, potentially a replacement for fine tuning. They introduce "trigger" tokens initialized to [MASK] tokens and then iteratively updated to maximize the label likelihood. Also, label tokens are selected from k-highest scoring word calculated by the current input. Liu et al. (2022b) proposes long propagation limits prompt tuning performances and then introduce prompt layer in the middle of LMs. It sets no gradient calculation after the prompt layer and hidden states before prompt layer are enough to generate a great prompts.

Prompt ensemble Prompt ensemble is a method that uses different kinds of prompt to generate a prediction which can bring all prompts' advantages together to improve and stabilize performance. It is a branch of ensemble learning Hansen and Salamon (1990) which has been a commonly used method to reduce the variance of the output.

The simplest way to do ensemble in prompt learning is using unweighted average. Jiang et al. (2020) first select top K highest performance prompt and then use their average probability for a single token at [Y] position. Lester et al. (2021) uses ensemble method in classification tasks. They first use different prompts for the same task and then obtain the result by choosing the majority of those output. The similar methods are widely used in other papers like Hambardzumyan et al. (2021) Qin and Eisner (2021a) created mixture modeling by giving each prompt's prediction a perturbation and learned it. Furthermore, they built a neural softmax model to learn the perturbation decided by both performance and the context of input. By this way the model can learn which prompt is most effective and form a ensemble model. Schick and Schütze (2020) combined knowledge distillation and ensemble learning. They trained a model for each manually -made prompt and use them to annotate another unlabeled dataset. A final classifier was trained in this dataset which can be recognized as the ensemble of all prompt. Peng et al. (2022) created an attention-based prompt ensemble model. They gave pre-trained model different

SST-2	
Single Prompt	
It was <mask>.	62.15
It's <mask>.	65.12
A <mask> read.	64.09
A <mask> one.	66.12
A <mask> one!	64.41
A <mask> piece.	63.04
All in all <mask>.	65.23
Ensemble Prompt	
All <mask> one.	67.54
ALL <mask> piece.	69.41

Table 1: Comparison of Single and Ensemble Prompts

kinds of prompts and obtained many logits. Based on these logits, They used an attention module to calculate the importance weights for every prompt sources and use the weights to select and make a better average of prompt.

3 Methodology

Given a language model $\mathcal{L}$, a training set $\mathcal{D}_{train}$, we construct $\mathcal{T}$ a template, $\mathcal{M}(y)$ the label word (which is a project of the binary labels $(0, 1)$ to a set of vocabulary words $v \in \mathcal{V}$ in classification task), a optional description of the task. These three together becomes our prompt $\mathcal{P}$. We find the best $\mathcal{T}$, $\mathcal{M}(y)$, and $\mathcal{P}$ that gives our best accuracy on the testing set $\mathcal{D}_{test}$. We describe each in detail in Appendix A. In our case, we first used a manual prompt to test a prompt-based zero shot as baseline setting, and in-context learning for GPT in Appendix A. Then we constructed discrete prompt on our own in Section 3.1 and lastly in Section 3.2 to show our result on ensemble using continuous prompt.

3.1 Discrete Prompt

In our study, we discovered that the structure of discrete prompts plays a significant role in assisting language models in downstream tasks, as evidenced by manual prompts and related literature. Different structures of discrete prompts demonstrate varying effectiveness in sentiment analysis across different sentence types. This underscores the necessity for a diverse and adaptive ensembled prompt to improve performance across a range of sentence structures and types.

Our approach takes inspiration from the Boosting concept and various methods of combining

prompts, also the framework of pre-trained models in the Dohan et al. (2022). Here, larger models were used to generate prompt, filter prompts and summary prompt. Then we conducted tests, and based on the results, we reduced the dataset and altered the template to produce new prompts.

T5-large The T5-large model is a transformer model pre-trained on a multi-task mixture of supervised and unsupervised tasks, offering a flexible approach to numerous language tasks. In our context, T5 is utilized to generate prompts automatically, relying on the provided training dataset and template.

Sentence BERT Sentence BERT, an extension of the BERT model, is designed for sentence-pair regression tasks such as semantic textual similarity (STS). It translates the prompt into an embedding space where the similarity with the word "sentiment analysis" can be calculated, providing a semantic comparison between prompts and the target sentiment.

Cosine Similarity Cosine similarity is a metric used to measure how similar the documents are, irrespective of their size. In our context, it is used to compute the similarity between the embeddings of the translated prompts and the term "sentiment analysis", providing a quantitative measure of their similarity.

BART BART is a transformer-based model, designed for denoising pre-training for natural language generation, translation, and summarization. In our approach, BART is used to summarize all prompts, providing a summarization that captures the overall information of the input prompts.

Pipeline The pipeline consists of four main parts: 1. Prompt Generation: The T5-large model is used to generate prompts automatically based on the training dataset and the provided template. Template like

$$< extra_i d_0 > great < extra_i d_1 > \quad (1)$$

$$great < extra_i d_0 > < extra_i d_1 > \quad (2)$$

$$< extra_i d_0 > < extra_i d_1 > great \quad (3)$$

2. Prompt Filtering: The generated prompts are translated by using Sentence BERT, and their cosine similarity with the word "sentiment analysis" is calculated. Prompts that do not meet a certain similarity threshold are eliminated.

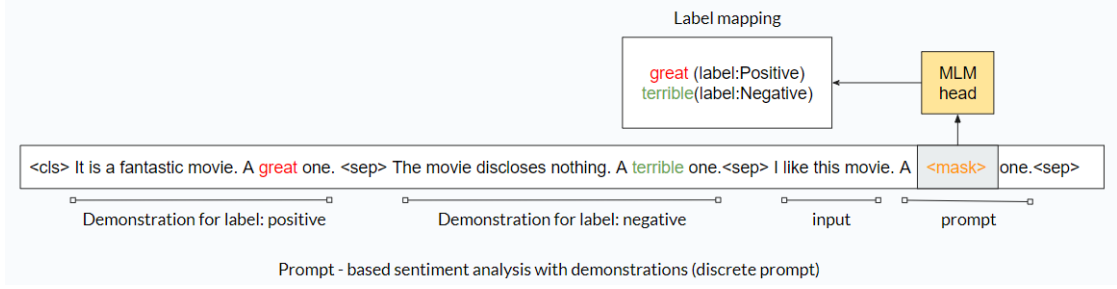


Figure 1: Framework of prompt-based sentiment analysis with demonstrations

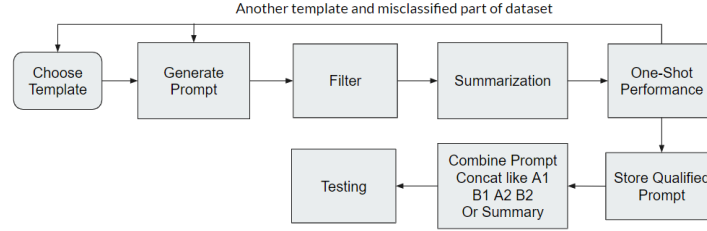


Figure 2: Main Idea of Discrete Prompt Ensemble: 1) Using T5-large model to generate prompt with various template. 2) Filter prompt by cosine similarity 3) Summarize prompts by BART model 4) Test the performance of prompt in training dataset

3. **Prompt Summarization:** After the filtering process, the BART model is used to summarize all remaining prompts, condensing them into a more manageable size while maintaining their overall meaning. Due to the input length limitation of the BART model, in our practical operation, we randomly select K prompts from the filtered prompts for summarization. This ensures that we can effectively utilize the summarization capabilities of the BART model without exceeding its input length constraints, thereby maintaining the integrity and relevance of the summarized output.

4. **Testing:** The selected prompt is fed into the RoBERTa-large model. The model’s performance with the prompt on the training dataset is evaluated and recorded. Incorrectly classified training data is noted and then used to generate the next round of prompts with a new template

3.2 CoPE

We build our continuous prompts, which we call **CoPE (Continuous Prompt Ensemble)**, an ensemble model under two assumptions: 1) There exists a better prompt in embedding space that cannot be represented by specific word tokens. 2) A single model can reach the sub-optimal solution and ensemble can outperform a single model. We propose two external architectures with two separate training phases (Figure 3): prompt encoder and

ensemble attention. Prompt encoder is used to generate continuous prompts and ensemble attention is the ensemble model of the logits derived from base prompting method.

Prompt Encoder We adopt the continuous prompt generation from P-tuning (Liu et al., 2021). In order for better association of continuous prompts, the prompt encoder is structured with two bidirectional LSTM layers and a multi-layer perceptron at the end. It generates continuous prompts $h_0, h_1, \dots, h_n$ by feeding in *pseudo tokens*. It will be concatenated with the embeddings of $\mathcal{X}$ and [MASK] in two types of templates:

$$\langle p \rangle e(\mathcal{X}) \langle p \rangle [\text{MASK}] \langle p \rangle \quad (4)$$

$$\langle p \rangle [\text{MASK}] \langle p \rangle e(\mathcal{X}) \langle p \rangle \quad (5)$$

where e represents the embedding of raw $\mathcal{X}$ instance and $\langle p \rangle$ represents a sub-section of the entire continuous prompts and the length of each sub-section can be modified. The changeable length and two types of templates enable the encoder generate diverse prompts. [MASK] is remained for pre-trained LMs to fill in a label word. We fix label word selection that a certain word represents a specific category. The pre-trained LM is frozen and only the prompt encoder is updated during training.

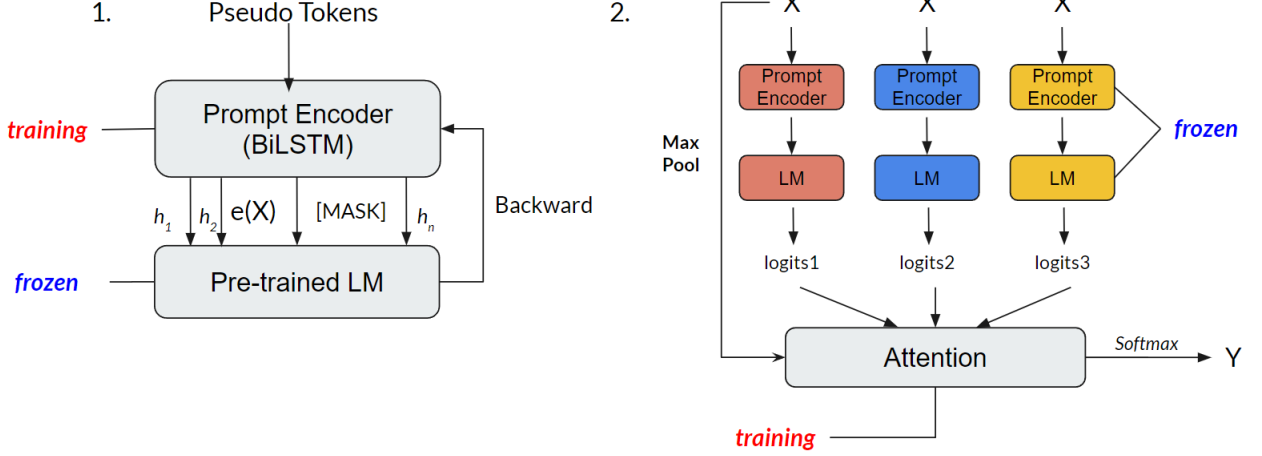


Figure 3: Two separate training phases of CoPE: 1) Prompt encoder generates continuous prompts concatenating with the embedding of $\mathcal{X}$ and [MASK]. Pre-trained LM is frozen and only the prompt encoder is updated during training. 2) Attention is trained separately. Attention takes logits from diverse base models and max-pooled $\mathcal{X}$ to calculate attention scores. Then logits are weighted summed to construct l_x .

Ensemble Attention¹ Since we’ve already have diverse prompt encoders, prompt encoders and pre-trained LMs are frozen and attention layer is updated during this training phase. The attention is used to generate weights to ensemble the base model’s pre-softmax logits. We adopt the methodology from SESoM(Peng et al., 2022). We max-pool the word embedding of an instance $\mathcal{X}$ and derive $\hat{X}$ as the representation of $\mathcal{X}$. Also, l_j represents the pre-softmax logits derived from the j -th model. $\hat{X}$ and l_j are fed into two separate feed forward networks with learnable weights. The outputs are annotated as h_x and $h_{l,j}$. Then we apply batch norm (Ioffe and Szegedy, 2015) to h_x . We compute attention score $s_{x,j}$ for l_j by

$$s_{x,j} = \frac{e^{(h_x \cdot h_{l,j})}}{\sum_{k=1}^T e^{(h_x \cdot h_{l,k})}} \quad (6)$$

Then, attention score $s_{x,j}$ is used to weighted sum l_j and the final output logit is derived by

$$l_x = \sum_{j=1}^T s_{x,j} h_{l,j} \quad (7)$$

With the few-shot setting, attention is only trained with $K=16$ (per class) samples by minimizing its cross-entropy loss between softmax l_x and Y .

4 Experiment

We use RoBERTa-large (Liu et al., 2019) for both discrete and continuous prompting.

¹In all of our experiments, our Num. Model = 3

4.1 Discrete Prompt with T5

Dataset We use SST-2 dataset to conduct sentiment classification and evaluate on accuracy.

Training setup We list three different templates for prompt generation and filter unqualified prompt whose cosine similarity score < 0.3 . We selected top $K=2$ (per class) relevant samples from training dataset as demonstration by calculating cosine similarity.

4.2 CoPE

Dataset We conduct our experiment on sentiment classification. We use SST-2, SST-5, CR, MR, Subj, and MPQA and evaluate on accuracy.

Training setup We propose three different types of prompts for base models. Since LM is prompt- and data- sensitive, prompts type selection varies for different datasets. Base models they are trained on full training dataset. We fix the epoch of prompt encoders to 15 since we want the base model to reach sub-optimal. Since attention layer is trained on few-shot setting, we randomly sample from training dataset with $K=16$ and $K=32$ (per class). We fix the epoch of the attention to 1 (We recognize more epochs do not lead to improvement). We directly adopt label word selection from LM-BFF(Gao et al., 2020). We apply Adam optimizer with learning rate of $1e-5$ and $3e-4$ (depending on datasets), weight decay of $5e-4$, and learning rate decay of 0.98. We use different seeds and record the one has the best performance.

$K=16$ (per class)	SST-2	SST-5	CR	MR	Subj	MPQA
Full Fine-tuning	95.00	58.70	89.40	90.80	97.00	87.80
Prompt 1	95.22	47.38	92.50	90.70	91.15	87.35
Prompt 2	95.44	49.05	90.15	90.05	92.30	82.55
Prompt 3	94.51	49.50	89.75	88.65	92.10	69.10
Average	94.86	48.44	90.90	89.68	91.63	78.23
CoPE	96.05	52.72	92.30	90.10	92.35	86.10

(a)

$K=32$ (per class)	SST-2	SST-5	CR	MR	Subj	MPQA
Full Fine-tuning	95.00	58.70	89.40	90.80	97.00	87.80
Prompt 1	94.62	52.94	91.60	84.10	84.95	87.35
Prompt 2	94.73	51.09	73.10	79.70	92.20	82.55
Prompt 3	94.62	51.18	76.70	53.75	92.10	69.85
Average	94.66	51.74	80.47	72.52	89.75	79.92
CoPE	95.17	52.35	85.80	81.70	91.65	86.05

(b)

Table 2: Our experiment result on RoBERTa-large. We conduct the experiment under several seeds and record the best one. We propose few-shot setting of $K=16$ and $K=32$ (per class) as comparison. Full fine-tuning is considered as our upper bound. Prompt 1, prompt 2, and prompt 3 represent three base prompt models. These three prompts may be different for different datasets. Also, the average accuracy among three base prompt models is provided for checking if our **CoPE** is effective.

Ensemble with Other Methods As a baseline, we also tried ensemble using mean of the logits, using a Gated Network (Joseph and Raj, 2023) that specific learn the logits weights, and using a separate Mixture Models similar to Qin and Eisner (2021b). We will show in our experiment that these method does not make significant different, thus we stick with the attention method.

	SST-2
prompt 1	94.84
prompt 2	95.06
prompt 3	95.44
CoPE	95.94
Gated Ensemble	95.66
Mean Ensemble	95.99

Table 3: Compare Two Other Ensemble Method. We see change the ensemble method for three models does not significantly change the accuracy, thus we only experiment with a single dataset.

Select Best K Our initial design is to choose the best K-shot at complete random. We also show choose few shot examples by cosine similarity, inspired by (Liu et al., 2022a). We use a sentence encoder (Cer et al., 2018) to encoder each example in $\mathcal{D}_{train}$ and $\mathcal{D}_{test}$. Then we calculate a pairwise

cosine similarity in the embedding space $e(\mathcal{D}_{train})$ and $e(\mathcal{D}_{test})$. We find the most similar sentence for each example in the $\mathcal{D}_{test}$, and count the occurrence of each example in the test set show as most similar. We then choose the most frequent top k example in this method. Further, we can select the overall best k, and the best k for a balance class (Table 4).

More shots As show in Table 2, we start with test a set of $K=16$ and $K=32$ for all benchmark dataset. In comparison, adding more shot to the ensemble model does not help with higher accuracy. Similarly, we tried $K=64$ and using entire testing set in the ensemble.

5 Result & Discussion

5.1 Discrete Prompt Ensemble

Result Analysis Given the computational resources and input limitations of the RoBERTa-large model, we only tested the effect of discrete prompts on the SST2 dataset where the number of examples is 2 per class. As shown in Table 1, the ensemble prompt performs slightly better than the single prompt. Also, under the condition of 2 examples per class, our prompts perform better than those in LM-BFF. However, as the RoBERTa-large model can only input 512 tokens, it cannot input 16 exam-

SST-2				
prompt 1	95.72			
prompt 2	95.06			
prompt 3	95.44			
CoPE	Three Runs			Avg
random k=32	95.99	96.10	96.05	96.05
most similary k=32	95.99	96.05	96.05	96.03
most similar balanced k=32	95.94	95.77	95.83	95.85

Table 4: Select best k. We show these three method are very similar thus is very hard to say which is the best shot-selecting method.

ples per class like LM-BFF. We will address this issue in future work. Notably, we observed that the language model is highly sensitive to the structure of discrete prompts - even slight changes can lead to significant variations in model performance.

Limitation Our study also encountered several limitations. Firstly, the discrete prompt ensemble was found to be highly unstable and demanded significant computational resources. Secondly, there is a lack of a stable, efficient method for prompt ensembling. Lastly, when using discrete prompts to assist the model in sentiment analysis, the quantity of examples significantly influenced the model’s effectiveness. Future work aims to address these limitations and further improve the stability and performance of the model.

5.2 Continuous Prompt Ensemble

Result Analysis As shown in Table 2, we set full fine-tuning as our upper bound. We show the base prompt performance and the ensemble performance to show a comparison. For SST-2, SST-5, and Subj, the ensemble outperforms each single prompt. For CR, MR, and MPQA, a single prompt can perform well enough and ensemble performance is vitiated by some other weak prompts. Surprisingly, some performance even outperforms our upper bound-full fine-tuning: **CoPE** of SST-2 under both few-shot settings, prompt 1 of CR under $K=32$ setting, and all performance CR under $K=16$ setting. Sometimes, our continuous prompts and **CoPE** behave unstably for some datasets. We conclude that RoBERTa-large is sensitive for prompts and datasets. Also, we propose two few-shot settings in order to analyze the effect of K . As a result, we do not recognize a clear improvement by increasing K .

We show in Table 3 that prompts are very unsta-

ble and very sensitive to the start points. If we don’t start with good soft prompt for the logits, we will likely loose performance for the ensemble model.

For testing with different ensemble method, similarly, we found that prompts are very sensitive to the input logits. This also gives us to conclude that using 3-models to ensemble does not gives enough to compare between ensemble models. More details in Appendix A.

For select best-K, we do not observe obvious advantages to choosing the few-shot examples by random. We hypothesis because we have limited computation resources, thus limit our model = 3 in the ensemble, it gives limit effect on the few-shot example. Thus, Shown in Table 4 we conclude that the best shot is at $K=16$.

Limitation After we change seeds several times, we recognize there is a latent upper bound performance for each dataset. On the one hand, if a single prompt cannot reach the upper bound, the ensemble will outperform each single prompt and reach the upper bound. On the other hand, if a single prompt is good enough to reach the upper bound performance, the ensemble cannot surpass the performance limit and behave like averaging single model’s logits, resulting in a sub-optimal performance. It could be the limitation from prompting itself, RoBERTa, or our ensemble method.

6 Future Work

6.1 Discrete prompt ensemble

Addressing Input Limitations The RoBERTa-large model’s input limitation of 512 tokens restricts the number of examples that can be input, unlike models in LM-BFF. We aim to address this limitation in the future, potentially by developing methods to effectively summarize or compress the input without losing critical information.

Developing Stable Ensemble Methods Our current research highlighted instability as a significant limitation in the ensemble of discrete prompts. Future work will focus on developing more stable methods for prompt ensembling, potentially exploring weighted ensembling or other techniques to reduce instability.

Optimizing Computational Efficiency The computational burden of working with discrete prompt ensembles is another area for improvement. We are going to develop more efficient algorithms or approaches to handle large volumes of prompts without compromising on performance.

Expanding Testing dataset Due to the limitations of the RoBERTa-large model, our current testing was restricted to the SST2 dataset with 2 examples per class. We will test the effectiveness of discrete ensemble prompts on larger datasets and with a greater number of examples per class.

6.2 Continuous prompt ensemble

Technical Issue Due to time and computational resource issue, we only record the best performance on several random seeds. It includes randomness in our experiments. Future work should be conducted more rigorously by taking the average performance on certain number of seeds and recording the standard deviation. Plus, we only propose three base models to do ensemble since they reach the GPU memory limit. If more GPU memory is allowed, more base models should be tried to see if ensemble is literally effective. Moreover, templates (4 and 5) are flexible to modify as well as hyper-parameters in training settings and structures in prompt encoders and the attention layer. There might be a better combination of them of all to reach a better performance, but more trials are needed. Or, there could be a unified method to derive the best combination.

Limitation As we mentioned in 5.2, more work needs to be done to analyze which part limits the performance. Using larger pre-trained LMs may be helpful; ensemble techniques can be improved (e.g. boosting); prompting with more tricks added may break through the limits.

7 Conclusion

In this project, we have shown that continuous prompts provide more flexible input by enabling

better prompts in embedding space, instead of being limited to a fixed set of options. Using prompts represented in embedding space rather than specific word tokens allows for more nuanced and effective guidance of a language model. Additionally, we proposed a new ensemble model called **CoPE**, (**C**ontinuous **P**rompt **E**nsible), which combines the strengths of multiple models to achieve better performance. By aggregating the predictions of different models, ensemble methods can reduce the impact of individual model weaknesses or errors. More importantly, our result shows that **CoPE** has enabled similar-size Roberta model to achieve competitive performance in natural language understanding tasks when compared to GPT model. This was previously thought to be impossible, but **CoPE** has opened up new possibilities for improving the performance of language models.

In conclusion, our study highlights the benefits of continuous prompts and ensemble methods for language modeling. These techniques can enhance the flexibility and accuracy of models, and have potential applications in a variety of fields. By addressing the limitations and refining these approaches, we can continue to improve the performance of language models and enable more advanced natural language processing applications.

8 Contribution

Yiming Tang Continuous prompt encoder and attention ensemble. Experiment on all datasets.

Ruyuan Zuo Ablation study, early stopping, find best few-shot examples based on similarity, gated ensemble, mixture model ensemble, discrete prompting with GPT2/3.5.

NingHao He Experiment mainly for dataset MR, SUBJ. Adjust hyper-parameters based on model performance. Discussion and deliver midterm and final paper.

Zhaoqi Xiao Whole pipeline of discrete prompt generation and ensemble. Experiment on SST-2 dataset.

References

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020a. [Language models are few-shot learners](#).

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020b. [Language models are few-shot learners](#).

Daniel Cer, Yinfei Yang, Sheng-yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, Yun-Hsuan Sung, Brian Strope, and Ray Kurzweil. 2018. [Universal sentence encoder](#). *CoRR*, abs/1803.11175.

Christopher Clark, Mark Yatskar, and Luke Zettlemoyer. 2019. [Don’t take the easy way out: Ensemble based methods for avoiding known dataset biases](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4069–4082, Hong Kong, China. Association for Computational Linguistics.

David Dohan, Winnie Xu, Aitor Lewkowycz, Jacob Austin, David Bieber, Raphael Gontijo Lopes, Yuhuai Wu, Henryk Michalewski, Rif A. Saurous, Jascha Sohl-dickstein, Kevin Murphy, and Charles Sutton. 2022. [Language model cascades](#).

Tianyu Gao, Adam Fisch, and Danqi Chen. 2020. [Making pre-trained language models better few-shot learners](#).

Karen Hambardzumyan, Hrant Khachatrian, and Jonathan May. 2021. [WARP: Word-level Adversarial ReProgramming](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4921–4933, Online. Association for Computational Linguistics.

L.K. Hansen and P. Salamon. 1990. [Neural network ensembles](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(10):993–1001.

- Sergey Ioffe and Christian Szegedy. 2015. [Batch normalization: Accelerating deep network training by reducing internal covariate shift](#).
- Zhengbao Jiang, Frank F. Xu, Jun Araki, and Graham Neubig. 2020. [How can we know what language models know?](#) *Transactions of the Association for Computational Linguistics*, 8:423–438.
- Manu Joseph and Harsh Raj. 2023. [Gate: Gated additive tree ensemble for tabular classification and regression](#).
- Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. [The power of scale for parameter-efficient prompt tuning](#).
- Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022a. [What makes good in-context examples for GPT-3?](#) In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, pages 100–114, Dublin, Ireland and Online. Association for Computational Linguistics.
- Xiangyang Liu, Tianxiang Sun, Xuanjing Huang, and Xipeng Qiu. 2022b. [Late prompt tuning: A late prompt could be better than many prompts](#).
- Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. 2021. [Gpt understands, too](#).
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. [Roberta: A robustly optimized bert pretraining approach](#).
- Xiangyu Peng, Chen Xing, Prafulla Kumar Choubey, Chien-Sheng Wu, and Caiming Xiong. 2022. [Model ensemble instead of prompt fusion: a sample-specific knowledge transfer method for few-shot prompt tuning](#).
- Guanghui Qin and Jason Eisner. 2021a. [Learning how to ask: Querying lms with mixtures of soft prompts](#).
- Guanghui Qin and Jason Eisner. 2021b. [Learning how to ask: Querying LMs with mixtures of soft prompts](#). In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5203–5212, Online. Association for Computational Linguistics.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019a. [Exploring the limits of transfer learning with a unified text-to-text transformer](#).
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019b. [Exploring the limits of transfer learning with a unified text-to-text transformer](#).
- Nils Reimers and Iryna Gurevych. 2019. [Sentence-BERT: Sentence embeddings using Siamese BERT-networks](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China. Association for Computational Linguistics.
- Laria Reynolds and Kyle McDonell. 2021. [Prompt programming for large language models: Beyond the few-shot paradigm](#).
- Timo Schick and Hinrich Schütze. 2020. [Exploiting cloze questions for few shot text classification and natural language inference](#).
- Taylor Shin, Yasaman Razeghi, Robert L. Logan, Eric Wallace, and Sameer Singh. 2020. [Autoprompt: Eliciting knowledge from language models with automatically generated prompts](#).
- Junjie Ye, Xuanning Chen, Nuo Xu, Can Zu, Zekai Shao, Shichun Liu, Yuhan Cui, Zeyang Zhou, Chao Gong, Yang Shen, Jie Zhou, Siming Chen, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023. [A comprehensive capability analysis of gpt-3 and gpt-3.5 series models](#).

A More on Prompt

Template and Description Take an example template,

$$t \in \mathcal{T} = \text{It was [MASK]}$$

Our template $\mathcal{T}$ is automatically generated by a language model, T5. In our experiment, we also found that provide a useful demonstration helps improve few-shot accuracy.

Demonstration We perform experiments on either *random sample* or *cosine similarity* way of selecting demonstrations from training set. In *random sample*, we found n random positive and negative example from the train set to concatenate as the demonstration. In *cosine similarity*, we convert the word embedding of sentence and calculate the cosine similarity between examples to find the best (details below)

B Sensitive Prompt

We show in Table 5 and Table 6 that prompt are extremely sensitive.

Model	SST-2 (acc)
RoBERTa-Large	33.00
BERT-Large	52.90

Table 5: Result using fixed template varies demonstration $n=2$. Template = It was [MASK]

Model	Datasets
(avg. acc of 7 template)	SST-2
RoBERTa-Large	41.57
BERT-Large	53.00

Table 6: Result using random sample demonstration, majority voted template

C Compare with GPT-3.5

Table 7 shows our result with comparing GPT3.5

Model	SST-2 (acc)
GPT-3.5 (manual prompt)	92.30
GPT-3.5 (random sample prompt)	81.98
BERT-large-cased (manual prompt)	79.13
RoBERTa-large	75.90

Table 7: Result using prompt-based zero-shot and in-context learning prediction baseline, using label words (terrible/great)